软　件　工　程

实验三：软件实现与构建

校园二手交易系统的编码实现、静态检查与仓库托管

课程名称　软件工程

项目名称　校园二手交易系统（CampusTrade）

学　　号　241220107

姓　　名　黄　毅

学　　院　计算机科学与技术学院

专　　业　计算机科学与技术

任课教师　王　豫

完成日期　2026 年 11 月

依据实验一 19 条需求与实验二 6 张 UML 图实现
Python 3，22 个源文件、3311 行，73 个单元测试
远程仓库 APY-tech/Software-Engineering

目录

1 实现概述 **1**

2 从 UML 图到代码 **2**

3 大模型辅助开发 **7**

4 代码风格与静态检查 **9**

5 编译与运行结果 **10**

6 Git 远程代码管理 **13**

1 实现概述

本次实验把实验二的设计落成可运行的程序：10 个领域类、7 个商品状态、19 条需求。语言用 Python 3.13，只用标准库，界面是命令行菜单。

选 Python 是因为这套系统的难点在规则而不在界面：同一商品至多一条有效预约、审核期间商品仍可被预约、已发布商品改内容要” 前台显示旧信息、后台审新内容”。这些规则的组合靠人工枚举容易漏，写成测试逐条撞最省力，`pylint`、`pytest` 也能直接用于后续实验的静态分析与测试。只用标准库则保证机房和自备机器上结果一致，不用处理依赖差异。

不做图形界面是权衡后的决定。实验二已给出 10 张界面稿，但本次重点是规则正确性，控制台菜单已能把全部用例走通。界面层与业务层完全分离，以后换 Tkinter 或 Web 只需替换 `campustrade/cli` 目录，服务层不用改。

代码分五层，依赖单向向下，界面层不直接碰仓储（表 1-1）。装配集中在 `context.py` 的 `build_application()`：先建仓储与日志，再建认证，最后建六个业务服务。依赖顺序写在一处，服务之间不互相 `import`，测试里一行就能拿到整套服务。

表 1-1 代码分层与职责（行数含空行与注释）

层	文件	职责	行数
领域模型	`models.py`、`errors.py`、`state.py`	实体、枚举、异常层次、状态迁移表	513
基础设施	`storage.py`、`security.py`	内存表 + JSON 快照、口令散列与会话令牌	305
服务层	`services/`（9 个文件）	认证、商品、审核、交易、举报、统计、日志与通知	1343
界面层	`cli/shell.py`、`cli/tables.py`	菜单、输入校验、表格排版	602
装配与入口	`context.py`、`seed.py`、`demo.py`、`run.py`	依赖装配、演示数据、端到端脚本、参数解析	548
合计	22 个源文件	——	3311

实现时有四条约定贯穿始终：

1. **状态只有一处可写**。商品状态不对外暴露 setter，所有迁移都走 `apply_status()`，先查迁移表再赋值并递增版本号，非法迁移抛 `Conflict`。
2. **权限判断跟着服务走**。每个面向界面的方法第一个参数都是会话令牌，方法内部换成 `User` 再判断角色与归属，界面层不可能” 忘记” 校验。
3. **持久化用整库快照加原子替换**。写临时文件后 `os.replace()`，进程中途被杀也不会留下半截 JSON。作业的数据量在千条级，这个方案比引入数据库省事，也便于人工查看。

4. **并发只在必要处加锁**。仓储持有一把可重入锁，`reserve()` 与 `cancel/confirm` 在临界区内完成” 查重—校验—写入”，其余操作不加锁。锁的范围就是” 同一商品至多一条有效预约” 需要保护的范围。

2 从 UML 图到代码

2.1 转换规则

先定规则再写类，每种 UML 元素只对应一种写法，后面 10 个类就是重复劳动（表 2-1）。

表 2-1 UML 元素到 Python 构造的转换规则

UML 元素	代码构造	处理要点
类（实体）	`@dataclass`	字段名、类型照抄；主键统一暴露 `ident` 属性，仓储据此泛型存取
枚举	`class X(str, Enum)`	继承 `str` 便于 JSON 落盘与命令行按名字解析
关联（1—N）	外键字段 + 仓储查询	不在实体里挂对象引用，避免循环引用与反序列化困难
组合（Item—ItemRevision）	独立表 + 按 `itemId` 查	待审内容能被单独丢弃，符合” 驳回即丢” 的语义
操作（方法）	服务层方法	原因见 2.2 节
状态图	迁移表 + `apply_status`	每条箭头一个表项，非法迁移抛异常
时序图	一个方法体 + 锁	消息顺序即语句顺序，`alt` 分支即 `if/else`

2.2 类图的落地与三处修订

领域模型（图 2-1）照实验二类图逐类翻译：10 个实体加 5 个枚举，全部落在 `models.py`（327 行）。实体只保留” 看自己” 的判断，例如 `Item.is_public()` 判断能否出现在公开查询里、`has_published_version` 判断驳回后是回草稿还是继续展示旧信息。图中 `ItemRevision` 是组合状态的载体：已发布商品被修改时 `Item.status` 仍是已发布，新内容暂存于待审记录，审核通过后原子替换，驳回则丢弃。`Reservation.requestId` 是幂等键，与商品编号一起保证同一商品至多一条有效预约。

服务层与仓储（图 2-2）是实验二类图里没有的，属于实现期新增的结构：`ServiceBase` 统一持有仓储、认证、日志三个依赖，六个业务服务继承它；`Store` 是仓储门面，`Table` 是泛型内存表，二者是组合关系。

实现过程中对原设计改了三处：

一是实体上的业务方法上移到服务层。实验二给 `Item` 挂了 `submitAudit()`、`offShelf()`，给 `Reservation` 挂了 `cancel()`、`confirm()`。真正写的时候发现这些方法都要读别的聚合——送审要看分类是否有效，取消预约要同时改商品状态——实

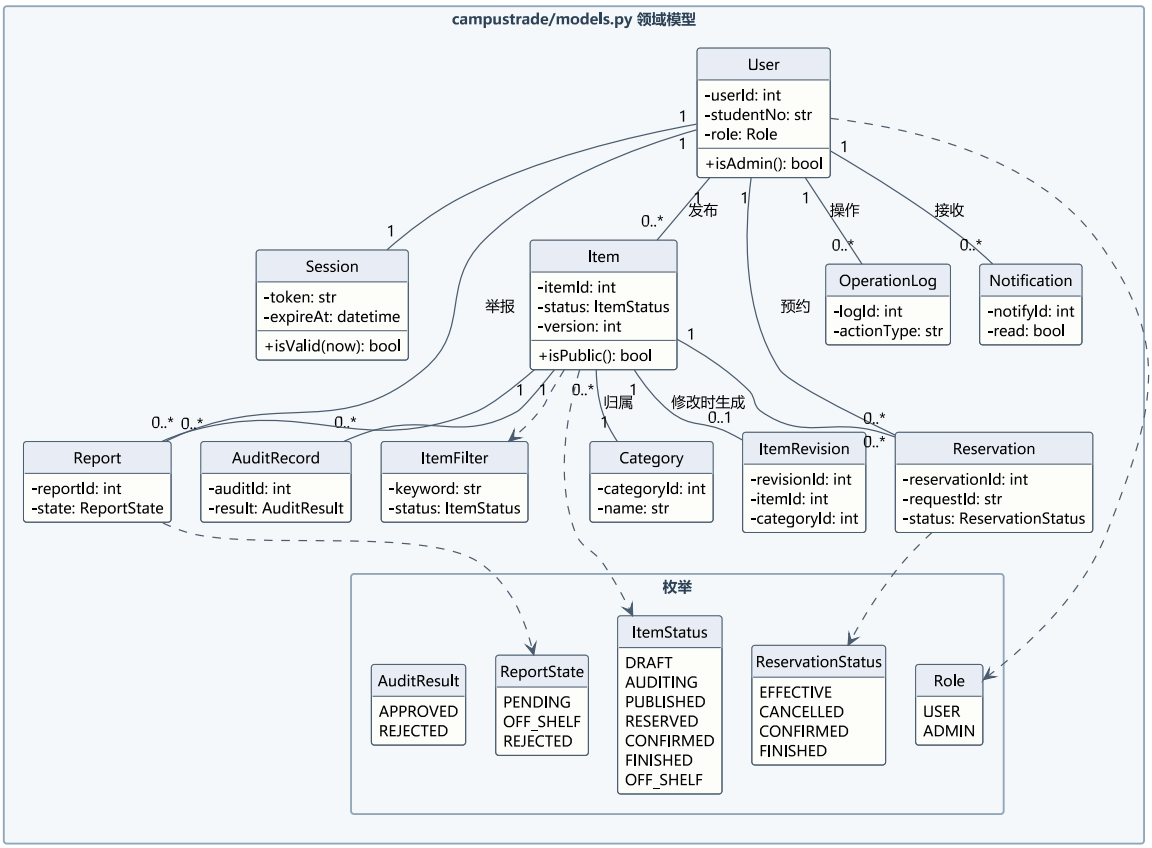


图 2-1　领域模型类图（矢量图，可放大查看）

体会被迫持有仓储引用，事务边界也就散了。改成”实体只描述数据与自身判断、服务负责跨实体协作”后，`Item.submitAudit()` 变成 `ItemService.submit_audit()`，状态迁移仍由 `state` 模块统一校验，规则没丢，只是换了执行者。

二是 `ItemRevision` 增加 `categoryId`。原设计只暂存标题、价格、图片，但 REQ-005 允许按分类筛选，改分类同样属于”修改商品信息”，不带上分类会导致审核通过后内容与分类不一致。

三是新增 `ServiceBase`、`Store`、`Table` 三个实现类。它们不是领域概念，是工程结构，分别收敛”令牌换用户与落盘””泛型持久化与查询”这几处重复代码。

2.3　状态图的落地

状态图（图 2-3）的 7 个状态、9 条合法迁移写成 `state.py` 里的一个字典，键是当前状态、值是允许的后继集合。`apply_status()` 先校验再赋值，并递增 `version`，这个版本号就是审核时”重新读取状态”的实现基础（REQ-003）。非法迁移一律走 `Conflict`，例如对已预约商品确认完成、对已完成商品再次下架。

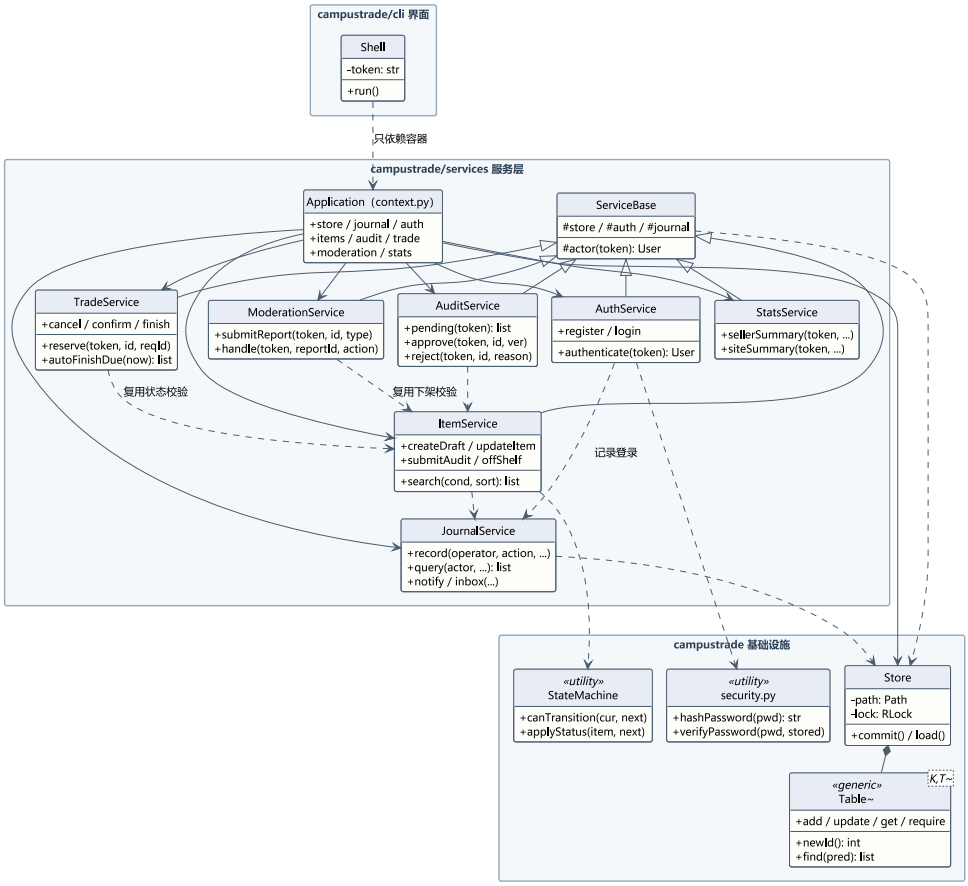


图 2-2　服务层与仓储类图

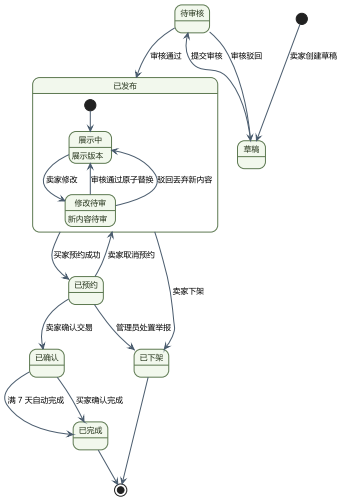


图 2-3　商品状态图（沿用实验二）

表 2-2 状态迁移与触发动作的对应

迁移	触发方法	附加校验
草稿 → 待审核	`ItemService.submit_audit`	标题、描述、价格必填；仅本人商品
待审核 → 已发布	`AuditService.approve`	仅管理员；版本号可比对
待审核 → 草稿	`AuditService.reject`	必须填写驳回原因；写审核记录
已发布 → 已预约	`TradeService.reserve`	锁内查重；非本人商品；幂等键
已预约 → 已发布	`TradeService.cancel`	仅卖家；预约记录置为已取消
已预约 → 已确认	`TradeService.confirm`	仅卖家；必须有有效预约
已确认 → 已完成	`TradeService.finish`	仅预约买家；满 7 天自动完成
已发布 → 已下架	`ItemService.off_shelf`	仅本人或管理员；已预约/已确认拒绝
已下架 → 已发布	`ItemService.restore`	仅管理员，卖家不能自行恢复

2.4 时序图的落地

预约是最容易出错的一条流程，按图 2-4逐条翻译：`alt` 的每个分支对应一个 `raise`，`with store.lock` 对应图上的临界区。查重放在最前，因为重发的请求必须返回原结果，此时商品已是”已预约”，若先校验状态就会把重发误判为冲突。

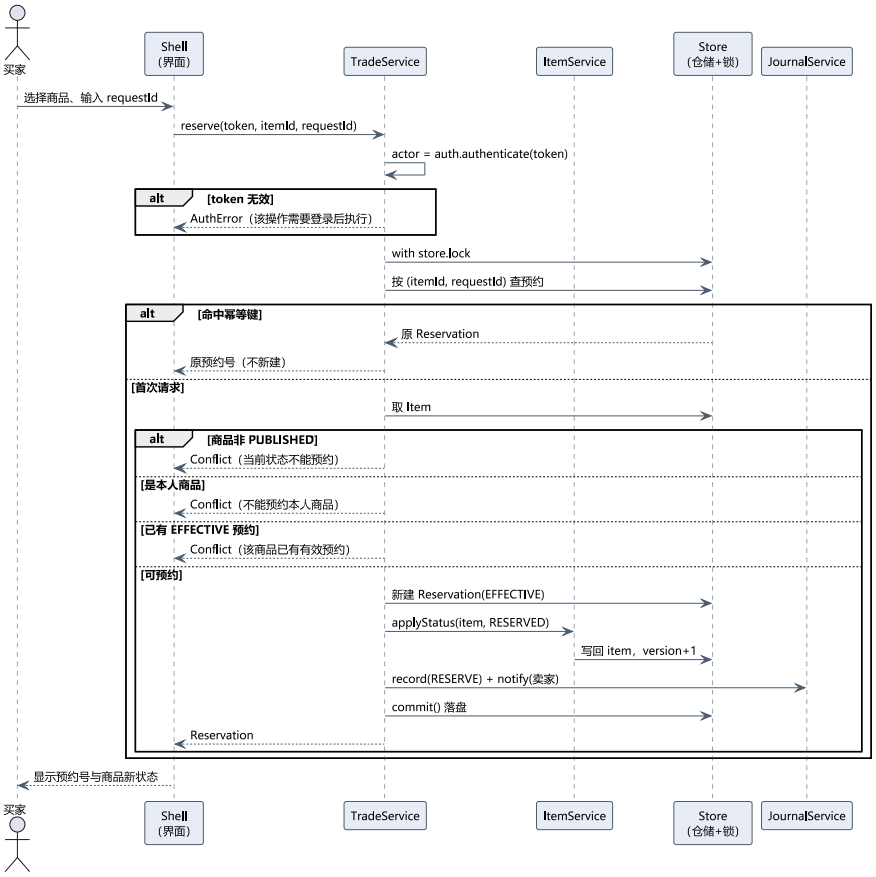


图 2-4 预约商品时序图（含幂等重发分支）

修改已发布商品是第二条关键流程（图 2-5）。它把”待审内容”与”商品状态”解耦：修改只写 `ItemRevision`，商品仍是已发布、前台仍显示旧信息，审核期间买家

照常预约；审核通过时在一个方法里替换全部字段并删除待审记录，驳回则只删待审记录。

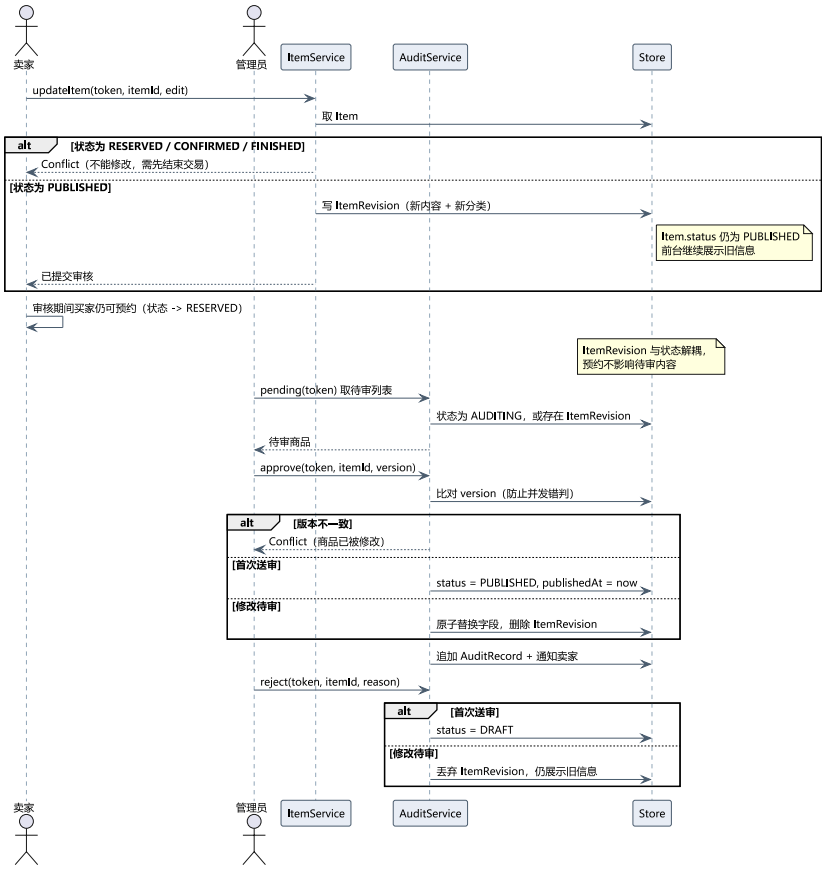


图 2-5　修改已发布商品与审核时序图

2.5　代码规模

最终交付 22 个源文件、3311 行（含空行与注释），去掉空行与整行注释后为 **2761 行**。另有 8 个测试文件、794 行、73 个用例（表 2-3）。其中服务层业务规则约 1186 行，占 43%。

表 2-3　源代码规模统计

分组	文件数	总行数	有效代码行	主要文件
领域模型	3	513	377	`models.py`（327）
基础设施	2	305	254	`storage.py`（260）
服务层	9	1343	1186	`item.py`（386）、`trade.py`（253）
界面层	3	602	500	`shell.py`（530）
装配与入口	5	548	444	`demo.py`（344）
小计	22	3311	**2761**	平均每文件 126 行有效代码
测试	8	794	577	`tests/`（73 个用例）

3 大模型辅助开发

3.1 使用方式

没有让模型一次把系统写出来。需求 19 条、状态 7 个，一次生成几百行代码，既没法逐行核对，也说不清哪条规则由哪段代码保证，出错只能整段重写。实际做法是按模块切分、先规约后实现、每轮跑检查，共十轮：

1. **一轮只喂一个模块**。输入是相关的 2–3 条需求原文、类图里对应那一段 PlantUML、以及已定下来的接口签名。模块小到能在一屏里核对，错了也只错这一块。
2. **先要存根与规约，再要实现**。先让它写方法签名、参数含义、前置后置条件、会抛哪些异常，确认规约对了再填函数体。规约写错的概率远低于实现写错，需求理解偏差在这一步就能暴露。
3. **每轮结束立刻跑三项检查**。`compileall` 查语法、`pylint` 查风格、`pytest` 查行为，把报错原文贴回下一轮让它按报错改，而不是由我转述错误。
4. **产出逐行过一遍**。每一行都读过再入库，改动与理由写进提交说明。

十轮的输入、产出与处理见表 3-1，完整提示词与被退回的版本在仓库 `code/docs/prompts.md`。

表 3-1 与大模型的十轮交互摘要

轮次	输入	模型产出	我的处理
R1	19 条需求 + 类图，只做数据模型	10 个 dataclass、5 个枚举、异常层次	采纳；补 `publishedAt`、`disputed` 两个字段
R2	仓储要能落盘、支持泛型表	`Table`、`Store`、编解码	换掉 `asdict`（Decimal/datetime 会丢类型），改原子写
R3	状态图 puml	迁移表 + `apply_status`	采纳；补中文状态名映射
R4	REQ-002/005/009/014/017	商品服务	**退回**：初稿直接改已发布商品的字段，改为写待审记录
R5	REQ-003/019	审核服务与审核记录	采纳；补版本号比对
R6	REQ-001/006/007/008/015	交易服务	**退回**：初稿无锁，改为锁内查重建单
R7	REQ-010/013/018	举报、日志、统计	采纳；处置改为复用商品下架
R8	全部服务	演示脚本、命令行菜单、表格排版	采纳结构；修 3 处残留与漏传令牌
R9	pylint 初稿 27 条告警	逐条修复方案	采纳 25 条，2 条按项目约定处理
R10	pytest 失败输出	定位到”完成只通知卖家”	采纳，改代码不改测试

3.2 模型承担了哪些工作

省时间的主要是三类活。一是**模板化代码**：dataclass 字段声明、异常类层次、`argparse` 参数定义，没有设计含量但手打容易漏。二是**把图翻成表**：状态图的 9 条箭头、时序图的 `alt` 分支，让它逐条列出" 箭头 → 触发方法 → 抛什么异常"，我再核对有无遗漏——这一项我最容易看走眼。三是**按告警给修复方案**：27 条 pylint 告警贴回去，折行、改 `types.NoneType`、多出口改单一出口，比自己一条条查文档快。

设计判断类的活它给不出可靠答案：锁加在哪里、要不要把方法从实体上移走、驳回后商品回不回草稿，都得自己定。它的默认倾向是" 能跑就行"，比如初稿愿意让已发布商品的修改直接覆盖字段，功能上似乎也对，但违反了 REQ-017 的展示语义。

3.3 生成内容不符合预期时怎么处理

办法是造反例：每条它声称的规则，都在演示脚本或测试里造一个应该被拒绝的场景去撞，撞不穿才算数。下面四个缺陷都是撞出来的，也都不在它主动说明的风险里。

案例 1：审核服务漏掉" 审核期间被预约" 的状态。演示脚本跑到第 7 节抛 `Conflict`：商品当前为已预约，不在待审核状态。初稿只认两种状态：

```
# 初稿：只认待审核与已发布
if item.status is ItemStatus.AUDITING:
    apply_status(item, ItemStatus.PUBLISHED)
elif item.status is ItemStatus.PUBLISHED and revision is not None:
    self._apply_revision(item, revision, now)
else:
    raise Conflict("不在待审核状态")

# 修改后：把"可挂待审内容的状态"抽成常量，三处共用
_REVISION_HOLDERS = (ItemStatus.PUBLISHED, ItemStatus.RESERVED,
                     ItemStatus.CONFIRMED)
elif revision is not None and item.status in _REVISION_HOLDERS:
    self._apply_revision(item, revision, now)
```

REQ-017 写明" 修改审核期间买家仍可预约"，商品会变成已预约，此时待审内容仍应能通过。这不是笔误，是模型没有把两条需求交叉起来想。补了 `test_revision_can_be_approved_after_reservation` 用例把它钉住。

案例 2：交易完成只通知卖家。`test_auto_finish_after_seven_days` 失败，买家收件箱里没有完成通知。REQ-015 原文是" 自动将交易标记为已完成**并通知买家**"，模型把" 完成" 当成只与卖家有关的事。判断是测试对、代码错，改 `_finish` 同时通知买卖双方，测试一行没动。

案例 3：必填校验放在建草稿时。初稿在 `create_draft` 就强制描述非空，写了一半的草稿存不下来，与" 草稿保存成功" 冲突。改成两档校验：建草稿只查标题、价格、分类，提交审核时再查描述。

案例 4：用 disable 消告警。storage.py 的编解码函数被判 too-many-return-statements，模型建议加 # pylint: disable。没有接受，改成” 先解 Optional、再按分支给 result 赋值” 的单一出口写法。告警消失是因为代码真的变简单了，而不是检查被关掉。

除此之外还退回了两种产出：把业务规则塞回实体方法的版本（规则散落、事务边界不清，见 2.2 节），以及” 先查再写” 不加锁的并发实现——两个线程可能都通过检查再各自写入，同一商品就会出现两条有效预约。

4 代码风格与静态检查

用 pylint 4.1.3 做静态检查，标准取 PEP 8（行宽 79、4 空格缩进），配置文件 pylintrc 放在代码根目录。只关掉三类与作业场景无关的检查：duplicate-code（模板化的 dataclass 声明必然相似）、too-few-public-methods（实体本来就只有数据）、too-many-lines。参数上限放宽到 6 个、属性上限 14 个，因为 Item 有 13 个字段，这是领域模型本身的样子，拆开反而难读。

第一版可运行代码（不含测试）跑出 **9.85/10**，共 27 条告警，分布见表 4-1。没有一条是逻辑错误类（E 级）告警，全是格式与写法问题——” 能跑” 和” 好读” 确实是两件事。

表 4-1 初稿 27 条告警的分布

告警	条数	典型位置	处理方式
line-too-long C0301	22	6 个文件，最长 83 字符	按语义折行，不缩排
trailing-newlines C0305	2	item.py、moderation.py	删除文件末尾多余空行
unidiomatic-typecheck C0123	1	storage.py:60	type(None) 改为 types.NoneType
too-many-return-statements R0911	1	storage.py:54	重写为单一出口
unused-import W0611	1	auth.py:12	删除未使用的 NotFound

行宽占 22 条，其中 15 条只超 1–4 个字符，删几个空格也能过，但那样只是把问题藏起来。按语义折行才是对的：函数调用的参数对齐到左括号后，if 的条件用括号包起来再按布尔项分行。

```
# 修复前（83 字符）
self._journal.record(actor.user_id, action, item.item_id, True, item.title)

# 修复后：按"调用参数对齐"折行
self._journal.record(
    actor.user_id, action, item.item_id, True, item.title)
```

把测试代码一起纳入检查后重新跑，得分 **10.00/10**：

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

测试目录有三处文件级 `disable`，都写了理由：pytest 夹具互相引用是官方写法（`redefined-outer-name`）、用例名本身就是文档（`missing-function-docstring`）、有的夹具只负责铺数据（`unused-argument`）。除此之外没有其它豁免，源码目录一条 `disable` 都没有。

这次也吃了晚检查的亏：第一次跑 pylint 是写完 3000 行之后，27 条里 22 条是行宽，属于写的时候就该注意的问题，攒到最后改就是一次几百行的机械劳动。

5 编译与运行结果

运行环境为 Windows 11 + Python 3.13.14（仅标准库），检查工具 pylint 4.1.3、pytest 9.2.0，UML 用 PlantUML 1.2024.8 渲染，报告用 TeX Live 2025 的 XeLaTeX 排版。

5.1 编译检查

Python 没有独立的编译步骤，这里用标准库 `compileall` 对全部源码做字节码编译，它会同时暴露语法错误与模块导入错误：

```
$ python -m compileall -q campustrade run.py tests
$ echo $?
0
```

没有任何输出、退出码为 0，说明 22 个源文件与 8 个测试文件都能通过语法与导入检查。运行时验证由 `python run.py --demo`（端到端）与交互式 `python run.py --seed` 两条路径覆盖。

5.2 单元测试

```
$ python -m pytest -q
```

```
.............................................................. [ 98%]
.                                                              [100%]
73 passed in 15.82s
```

测试按模块分文件，覆盖关系见表 5-1。没有追求行覆盖率数字，而是每写一条规则就配一个” 应该被拒绝” 的用例：越权、状态不允许、重复提交、时间未到，这些分支才是缺陷的藏身处。

表 5-1 测试文件与覆盖的需求

测试文件	用例数	覆盖内容
test_auth.py	12	注册、登录、重复学号、弱口令、会话过期、停用账号、改密码（REQ-004/012/016）
test_item.py	15	草稿、公开查询过滤与排序、必填校验、下架与恢复、分类唯一性与引用保护（REQ-002/005/009/014）
test_audit.py	10	通过/驳回、待审内容原子替换、驳回丢弃、版本冲突、审核历史与可见性（REQ-003/017/019）
test_trade.py	13	幂等重发、唯一有效预约、取消、确认、完成、超时自动完成、异议暂停、并发竞争（REQ-001/006/007/008/015）
test_moderation.py	8	举报去重、处置下架/驳回、重复处理、已预约商品不能下架（REQ-010/014）
test_storage.py	10	JSON 快照往返、时间范围统计、日志权限、损坏文件处理（REQ-013/018）
test_cli.py	5	登录、搜索、预约、非法选项、管理员菜单的界面冒烟
合计	73	——

5.3 端到端演示

`python run.py --demo` 在内存库上按需求编号跑 11 节场景，每节既有”应当成功”的路径，也有”应当被拒绝”的反例。下面保留正文点名的五节，其余六节省略（✓ 表示按预期成功，× 表示按预期被拒绝）：

```
=== 1. 注册与登录（REQ-004） ================================
  × 重复学号注册 -> 已拒绝：学号 241220101 已被注册
  × 密码错误登录 -> 已拒绝：学号或密码错误
  ✓ 登录态有效：241220101，角色 USER
  × 未登录直接预约 -> 已拒绝：该操作需要登录后执行

  ……（本节略）
  ……（本节略）
=== 4. 预约的幂等与唯一性（REQ-001） ======================
  ✓ 买家预约成功，预约号 1，商品状态 已预约
  ✓ 同一 requestId 重发返回原记录：预约号 1（共 1 条预约记录）
  × 另一买家再预约 -> 已拒绝：商品当前为已预约，不能预约
  × 卖家预约本人商品 -> 已拒绝：不能预约本人发布的商品
  × 缺少 requestId -> 已拒绝：缺少幂等键 requestId

  ……（本节略）
  ……（本节略）
=== 7. 修改已发布商品（REQ-017） ============================
  ✓ 修改已发布商品后，前台仍显示《罗技 M590 静音鼠标》/89.00 元，状态 已发布
  ✓ 新内容暂存于待审记录 #1，提交时间 21:28:23
  ✓ 该商品进入待审核列表：True
  ✓ 修改审核期间买家仍可预约：预约号 3，前台仍是旧标题《罗技 M590 静音鼠标》
  ✓ 审核通过后原子更新为 79.00 元，待审记录已清除：True
  ✓ 驳回修改后仍为已发布，显示原价 45.00 元，待审内容已丢弃：True

  ……（本节略）
=== 9. 并发预约（REQ-001 竞争场景） ========================
```

```
  √ 4 个买家同时预约，结果：['成功', '失败（CONFLICT）', '失败（CONFLICT）',
    '失败（CONFLICT）']
  √ 商品最终状态 已预约，有效预约 1 条

=== 10. 超时自动完成（REQ-015） ============================
  √ 把确认时间回拨 8 天，模拟时间流逝
  √ 自动完成 1 笔，商品状态 已完成
  √ 卖家共收到 16 条通知，最新一条：《护眼台灯》交易已完成，金额 35.00 元
```

四处结果值得单独说明：第 4 节同一 `requestId` 重发返回预约号 1，记录仍只有 1 条，另一买家再预约被拒；第 7 节修改已发布商品后前台仍显示旧标题与 89 元，期间买家仍能预约，审核通过后价格变为 79 元且待审记录被清除；第 9 节 4 个买家经屏障同步后同时预约，只有 1 个成功，其余 3 个收到 `CONFLICT`；第 10 节把确认时间回拨 8 天后触发自动完成，买卖双方均收到通知。

5.4 交互式运行

`python run.py --seed` 进入菜单界面。下面是真实会话，省去了每条命令后重复打印的菜单与输入提示，只在 → 后保留该步结果：管理员审核通过一件待审商品，买家搜索并预约，卖家确认交易，买家确认完成，最后卖家查看统计。

```
已初始化演示数据：管理员 900000001，密码 ********
================================================================
                  校园二手交易系统 • 命令行版
================================================================

  → 登录成功。

  当前身份：900000001（管理员）
  编号  标题                    状态    版本  待审修改
  ----  ----------------------  ------  ----  --------
     3  考研英语真题 2010-2024  待审核     2  无

  当前身份：900000001（管理员）
  → 审核通过，商品状态：已发布

  当前身份：900000001（管理员）
  → 登录成功。

  当前身份：241220102（买家/卖家）
  → 命中 3 件
  编号  标题                     价格  状态    发布
  ----  ----------------------  -----  ------  -----------
     3  考研英语真题 2010-2024  45.00  已发布  10-31 21:28
     2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
     1  高等数学 第七版 上册    18.00  已发布  10-31 21:28

  当前身份：241220102（买家/卖家）
  → 预约成功：预约号 1，请求号 req-cli-a

  当前身份：241220102（买家/卖家）
  预约号  商品  买家  状态  异议
  ------  ----  ----  ----  ----
```

```
     1    3    3  有效  否

当前身份：241220102（买家/卖家）
→ 登录成功。

当前身份：241220101（买家/卖家）
编号  标题                    价格  状态    发布
----  ---------------------  -----  ------  -----------
   3  考研英语真题 2010-2024  45.00  已预约  10-31 21:28
   2  罗技 M590 静音鼠标      89.00  已发布  10-31 21:28
   1  高等数学 第七版 上册    18.00  已发布  10-31 21:28
   4  待补图的书架            60.00  草稿    -

当前身份：241220101（买家/卖家）
→ 已确认交易，请与买家线下交付。

当前身份：241220101（买家/卖家）
→ 登录成功。

当前身份：241220102（买家/卖家）
→ 交易已完成。

当前身份：241220102（买家/卖家）
→ 登录成功。

当前身份：241220101（买家/卖家）
→ 发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元

当前身份：241220101（买家/卖家）
```

权限随身份变化：管理员登录后菜单多出 21–30 项（审核、举报、统计、日志、分类），普通用户只有 1–15 项，未登录只能浏览。最后一行”发布 4 件，预约 1 次，完成 1 笔，交易额 45.00 元”与前面几步操作对得上，说明统计没有把别人的数据算进来。

5.5 未覆盖的部分

有三点没有做到，如实列出：REQ-011 的”1 万条商品、20 并发、95% 查询 1 秒内”未做实测，单机内存表加整库快照的并发模型与 Web 服务不同，测出的数字不能直接对照，留待实验四做基准；REQ-012 的 HTTPS 在桌面程序里没有对应物，其中”密码加密存储、校验不还原明文”已实现（PBKDF2-HMAC-SHA256，12 万轮，随机盐）；图形界面没有落地，实验二的 10 张界面稿仍停留在原型，控制台能覆盖全部用例但演示效果差，这是本次最明显的短板。

6 Git 远程代码管理

本地仓库在 `repo/`，远程托管在 GitHub：

```
$ git remote -v
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
```

```
origin  https://github.com/APY-tech/Software-Engineering.git (push)
```

主分支 `main`，三次实验的产物都在同一仓库里，靠目录与提交说明区分，没有另开分支——交付物是持续演进的同一个项目，分支反而增加合并成本。实验一的两次关键状态另打了标签 `lab1-ai-draft` 与 `lab1-final`，便于对照初稿与定稿。

本轮工作按”一类产物一次提交”拆开（表 6-1）：代码与它的检查记录分开，报告单独提交，避免 3000 行代码和一份 PDF 混在一个提交里导致 diff 无法阅读。

表 6-1　本次实验的提交记录

提交	文件数	内容
a66e16a	34	22 个源文件、8 个测试文件、配置与说明
31056f2	8	pylint 前后对比、测试与演示输出、代码规模、提示词记录
ad62b25	12	类图、服务类图、两张时序图的 puml 与 png
e2e555f	20	报告 LaTeX 源、插图与编译产物

仓库最新的提交历史：

```
=== git log (main) ===
9745ea9 实验三 代码留档：git 记录补全到 HEAD，附实验三提交明细
75bb7a1 实验三 代码留档：补充 git 提交历史与远程地址记录
e2e555f 实验三 报告：实现与构建报告 26 页，含真实检查与运行输出
ad62b25 实验三 UML：按实现修订类图，新增服务类图与两张时序图
31056f2 实验三 代码留档：pylint 前后对比、测试与演示输出、提示词记录
a66e16a 实验三 代码：19 条需求的 Python 分层实现，22 个文件
09a2689 实验二 报告修订：表4-1 每条记录显式给出优缺点，表4-2 编号与列宽
cb3bdc6 新增 README：仓库结构、重新生成方式与提交约定
2b79a57 实验二 报告：5 章 17 页，新增 AI 工作流与 git 追踪两节
a16c181 回改实验一：REQ-017 驳回后状态、REQ-001 幂等键、REQ-009 模块归属
961f4e0 实验二 UI：10 个界面稿与可交互原型源文件
76f4651 实验二 UML：六张图的 PlantUML 源码与渲染图
9745ea9 实验三 代码留档：git 记录补全到 HEAD，附实验三提交明细
75bb7a1 实验三 代码留档：补充 git 提交历史与远程地址记录
e2e555f 实验三 报告：实现与构建报告 26 页，含真实检查与运行输出
ad62b25 实验三 UML：按实现修订类图，新增服务类图与两张时序图
31056f2 实验三 代码留档：pylint 前后对比、测试与演示输出、提示词记录
origin  https://github.com/APY-tech/Software-Engineering.git (fetch)
origin  https://github.com/APY-tech/Software-Engineering.git (push)
## main...origin/main
```

`.gitignore` 排除了三类东西：Python 的 `__pycache__/` 与 `.pytest_cache/`；程序运行产生的 `code/data/campustrade.json`（可用 `--seed` 重建，不占仓库体积）；LaTeX 中间产物（`.aux`、`.log`、`.toc`）。源码、UML 的 puml 与图、测试、检查记录、报告源与 PDF 都入库。